Experimentação — Sprint 5

Construção de um LLM do Zero · Pedro Devens Dellalibera e João Gabriel Trevisol

Nesta sprint o modelo GPT construído na Sprint 4 foi treinado pela primeira vez. Os experimentos cobrem os Entregáveis 2 a 6: a avaliação do modelo com cross-entropy e perplexidade (`5,1.py`), a rotina de treinamento (`5,2.py`), os experimentos de treinamento (`Experimentos.py`), as estratégias de geração de texto (`5,3.py`) e o salvamento e carregamento dos pesos (`5,4.py`). Como extra, carregamos os pesos do GPT-2 publicados pela OpenAI no mesmo modelo (`5,5.py`).

Todos os scripts salvam seus resultados em `relatório/Sprint 5/resultados/` (JSON, CSV e gráficos), e os números deste documento foram copiados de lá.

Metodologia

- **Configuração base:** a mesma `GPT_CONFIG_124M` da Sprint 4, com o `context_length` reduzido de 1024 para 256 tokens, como no livro. O modelo tem 162.419.712 parâmetros.
- **Dados:** o conto The Verdict (`data/the-verdict.txt`), com 20.480 caracteres e 5.146 tokens BPE. Os primeiros 90% dos caracteres vão para o treino e os 10% finais para a validação. Com janelas de 256 tokens sem sobreposição e lotes de 2, isso dá 9 lotes de treino (4.608 tokens) e 1 lote de validação (512 tokens).
- **Treino base:** AdamW com taxa de aprendizado 0,0004 e weight decay 0,1, 10 épocas, avaliação a cada 5 passos sobre 5 lotes (`eval_freq=5`, `eval_iter=5`), contexto inicial "Every effort moves you" e `torch.manual_seed(123)`, exatamente como na seção 5.2 do livro.
- **Medição por época:** além das avaliações a cada 5 passos, ao fim de cada época medimos a perda sobre os conjuntos inteiros de treino e validação e guardamos a amostra de texto. Essa medição usa uma cópia do loader de treino sem embaralhamento, para não mudar a ordem dos lotes.
- **Experimentos de treinamento:** os valores testados são os do grid do repositório do livro (`ch05/05_bonus_hparam_tuning/hparam_search.py`). Testar todas as combinações seria inviável, então mudamos um parâmetro por vez e mantivemos o resto igual ao treino base.
- **Geração:** cada estratégia gerou 30 tokens a partir do mesmo contexto, com 3 sementes (123, 456 e 789), no modelo sem treino e no modelo treinado. Medimos a porcentagem de tokens únicos (diversidade), de bigramas repetidos (repetição) e o maior trecho, em tokens, copiado literalmente do The Verdict (memorização).

Ambiente	Descrição
Execução	`GPU — NVIDIA GeForce RTX 3050 6GB Laptop (CUDA)`
Sistema	`Windows · Python 3.14 · PyTorch 2.15.0+cu134`
Modelo	`GPT_CONFIG_124M com context_length = 256 (162.419.712 parâmetros)`
Treino base	`AdamW (lr 0,0004; weight decay 0,1) · 10 épocas · batch 2 · seed 123`

Tempo total dos 8 scripts	`12 min 23 s`

Script utilizado

O `Experimentos.py` carrega o `5,2.py` com `importlib` (o nome tem vírgula e não pode ser importado com `import` comum, como já acontecia na Sprint 4) e reaproveita a função `treinar()`. Cada treino salva um JSON em `resultados/experimentos/` e é pulado se o JSON já existir, então a execução pode ser interrompida e retomada.

```
EXPERIMENTOS = {
    "epocas": [("25 epocas", {}, {"num_epochs": 25})],
    "lr":      [(f"lr={lr}", {}, {"learning_rate": lr})
                for lr in [0.0001, 0.0005, 0.001, 0.005]],
    "dropout": [(f"dropout={d}", {"drop_rate": d}, {})
                for d in [0.0, 0.1, 0.2]],
}
# python "src/Sprint 5/Experimentos.py"                    -> os tres grupos
# python "src/Sprint 5/Experimentos.py" lr                 -> so um grupo
```

Experimento 1 — Avaliação do modelo sem treino

- Cross-entropy loss e perplexidade (Entregável 2).

O que testar: Se a perda calculada passo a passo bate com a `cross_entropy` do PyTorch, e qual é a perda de um modelo que ainda não aprendeu nada.

Como fizemos: No `5,1.py`, usamos os dois exemplos do livro ("every effort moves" e "I really like"): softmax sobre os logits, probabilidade do token correto, logaritmo, média e troca de sinal. Depois calculamos as perdas sobre os loaders de treino e validação.

Métrica	Valor
Probabilidades dos alvos (texto 1)	$7{,}45\times10^{-5}$ · $3{,}11\times10^{-5}$ · $1{,}16\times10^{-5}$
Perda manual (média negativa das log-prob.)	10,793963
cross_entropy do PyTorch	10,793964
Perplexidade do exemplo	48.725,82
Perda inicial — treino	10,9876
Perda inicial — validação	10,9832
Perplexidade inicial — treino	59.135,30
Perplexidade inicial — validação	58.874,25
Referência: ln(50.257)	10,8249

As probabilidades dos tokens corretos ficam na ordem de 10^{-5}, perto de $1/50.257 \approx 2 \times 10^{-5}$, que seria um sorteio uniforme sobre o vocabulário. Por isso a perda inicial (10,9876 no treino) fica perto de ln(50.257) ≈ 10,82, e a perplexidade fica na ordem do tamanho do vocabulário: o modelo está "em

dúvida" entre praticamente todos os tokens. O cálculo manual e a função do PyTorch deram o mesmo valor até a sexta casa decimal, o que confirma que a `cross_entropy` faz internamente a softmax, o log e a média.

No livro, os valores de referência são 10,7940 no exemplo e 10,988 / 10,981 nas perdas iniciais. Os nossos (10,7940, 10,9876 e 10,9832) reproduzem esses valores, o que valida a implementação.

Experimento 2 — Treino base (configuração do livro)

- Rotina de treinamento e acompanhamento das perdas (Entregável 3).

O que testar: Se o modelo está de fato aprendendo a prever o próximo token, acompanhando as perdas de treino e de validação ao longo das 10 épocas.

Como fizemos: Rodamos o `5,2.py` com a configuração da seção 5.2 do livro. O script imprime as avaliações a cada 5 passos e, ao fim de cada época, as perdas sobre os conjuntos inteiros e uma amostra de texto.

Parâmetro	Valor
Épocas	`10 (9 passos por época, 90 passos no total)`
Tamanho do lote	`2 sequências de 256 tokens`
Otimizador	`AdamW · lr = 0,0004 · weight_decay = 0,1`
Avaliação	`a cada 5 passos, sobre 5 lotes`
Tempo total	`52,23 s (882 tokens/s)`

Época	**Perda treino**	**Perda validação**	**Diferença**	**Perplexidade val.**
1	7,1739	7,4714	0,2974	1.757,0
2	5,9153	6,5220	0,6067	680,0
3	5,5226	6,4052	0,8826	605,0
4	4,7610	6,2510	1,4900	518,5
5	3,7986	6,1540	2,3554	470,6
6	2,8833	6,1247	3,2414	457,0
7	2,0742	6,1791	4,1049	482,5
8	1,3742	6,2318	4,8576	508,6
9	0,8440	6,3117	5,4677	551,0
10	0,4823	6,3814	5,8991	590,8

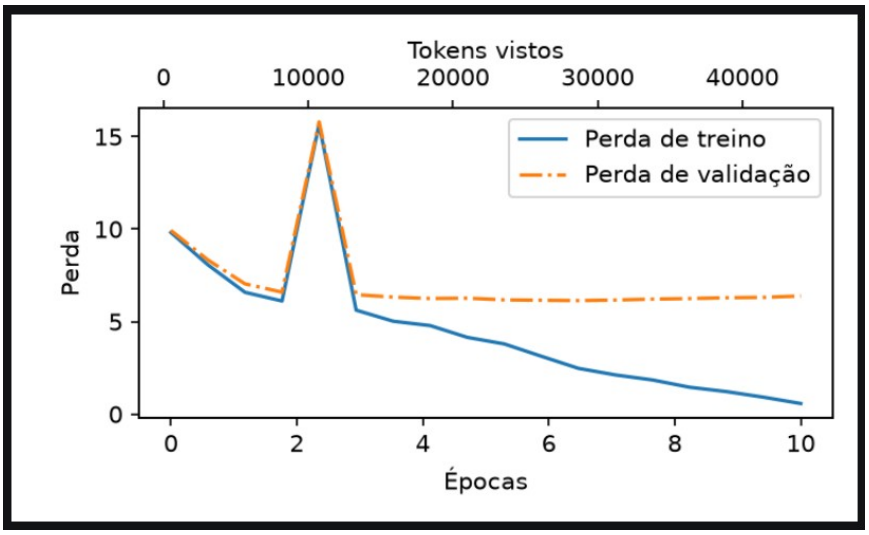


Figura 1 — Perdas de treino e validação a cada 5 passos (5_2_perdas.png).

A perda de treino cai do valor inicial de ≈ 11 até **0,4823** na época 10. A perda de validação cai junto nas primeiras épocas e depois para de melhorar na **época 6**, ficando em **6,1247** — a partir dali ela volta a subir. A distância entre as duas curvas, que começa em 0,2974, chega a **5,8991** no fim. É o padrão de sobreajuste descrito no livro, que reporta 0,569 no treino e 6,373 na validação na mesma configuração: os nossos 0,482 e 6,381 ficam a menos de 0,1 desses valores.

Um detalhe do registro a cada 5 passos: no passo 20 a perda salta para 15,67 (treino) e 15,80 (validação) e volta ao normal no passo seguinte. É um pico isolado de instabilidade do AdamW no começo do treino, que não aparece nas medições por época porque estas são feitas sobre os conjuntos inteiros no fim de cada época.

A evolução do texto gerado a partir de "Every effort moves you" mostra a mesma coisa:

Época	**Texto gerado (greedy, 50 tokens)**
1	`"Every effort moves you,,,,,,,,,,,,,."` seguido de dezenas de quebras de linha
3	`"Every effort moves you, and, and his. Gis. G, and. G. I had to the to the-- to the Gis."`
5	`"Every effort moves you know the" + quebras de linha + "a, and his the, and his of"`
10	`"Every effort moves you?" / "Yes--quite insensible to the irony. She wanted him vindicated--and by me!" / "He laughed again, and threw back his head to look up at the sketch of the donkey. "There were days when I"`

Nas primeiras épocas o modelo só repete vírgulas e palavras muito frequentes (*the*, *and*). No meio do treino ele já monta frases com a gramática do conto e, na época 10, reproduz um trecho literal e contínuo do The Verdict.

Experimento 3 — Número de épocas (5, 10, 15, 20 e 25)

- Diferentes números de épocas e sinais de sobreajuste (Entregável 4).

O que testar: Até que ponto vale a pena continuar treinando: em que época a perda de validação é mínima e o que acontece com o texto depois disso.

Como fizemos: Em vez de cinco treinos separados, fizemos um único treino de 25 épocas (o maior valor do grid) e lemos as perdas e o texto no fim das épocas 5, 10, 15, 20 e 25. Como a semente e a ordem dos lotes são as mesmas, as 10 primeiras épocas reproduzem o treino base.

Época	Perda treino	Perda validação	Diferença	Tempo acumulado
5	3,799	6,154	2,355	26 s
10	0,482	6,381	5,899	53 s
15	0,072	6,937	6,865	79 s
20	0,027	7,126	7,099	106 s
25	0,012	7,296	7,284	132 s

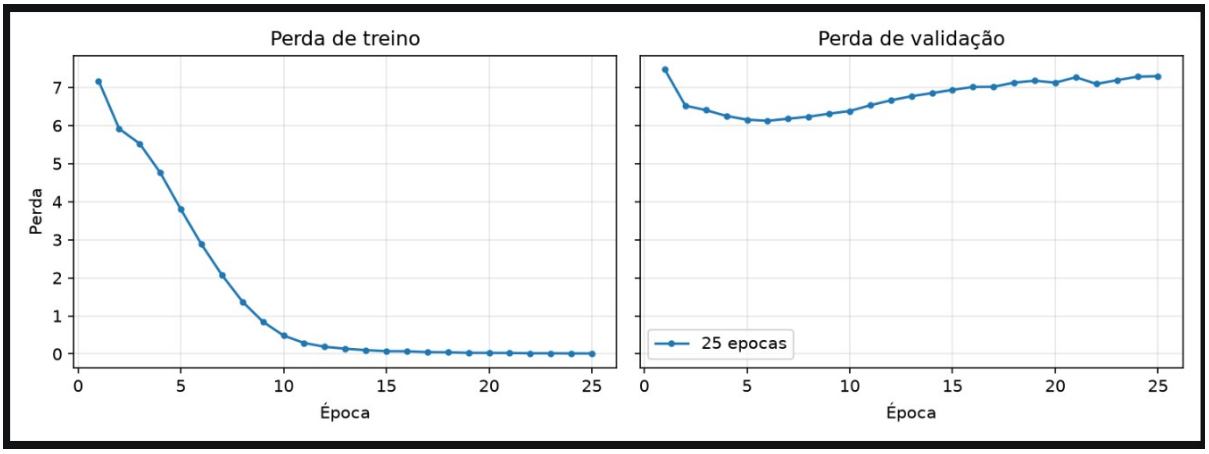


Figura 2 — Perdas por época no treino de 25 épocas (experimentos/epocas.png).

A menor perda de validação aparece na **época 6 (6,1247)**. Depois dela, a perda de treino continua caindo até **0,012** na época 25 — praticamente zero —, enquanto a de validação **sobe de forma contínua** até 7,296, pior que o valor da época 2. Cada época extra custa cerca de **5,3 segundos** e, a partir da época 6, só aumenta a memorização do texto de treino.

O texto gerado confirma: nas épocas 10, 15, 20 e 25 a saída em modo greedy é **exatamente a mesma**, palavra por palavra. Depois da época 10 o modelo não aprende mais nada novo sobre o texto — apenas aumenta a confiança no que já decorou, o que a perda de treino caindo para 0,012 mostra numericamente.

Experimento 4 — Taxa de aprendizado

- Diferentes taxas de aprendizado (Entregável 4).

O que testar: O efeito do tamanho do passo do otimizador na velocidade e na estabilidade da convergência.

Como fizemos: Quatro treinos de 10 épocas, mudando apenas `learning_rate` para os valores do grid do livro: 0,0001, 0,0005, 0,001 e 0,005.

Taxa	Perda treino final	Perda val. final	Melhor val. (época)	Tempo (min)
0,0001	2,869	6,252	6,238 (8)	0,9
0,0005	0,582	6,448	6,206 (7)	0,9
0,001	4,248	6,506	6,506 (10)	0,9
0,005	5,607	7,232	7,232 (10)	0,9

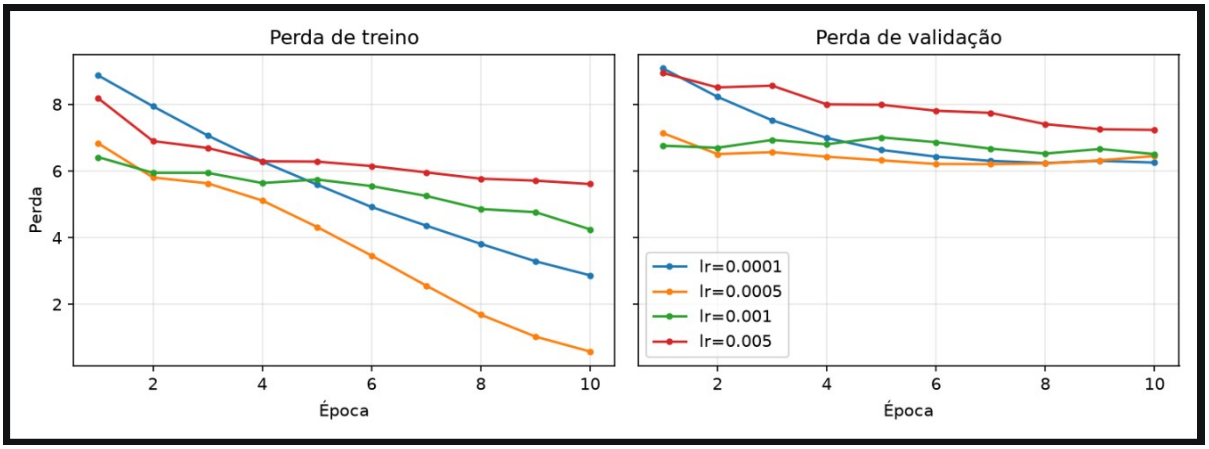


Figura 3 — Perdas por época para cada taxa de aprendizado (experimentos/lr.png).

Com **0,0001** o modelo converge devagar: depois de 10 épocas a perda de treino ainda está em 2,869, bem longe dos 0,482 do treino base. Em compensação, é a taxa com a **melhor perda de validação final** (6,252) — justamente porque não teve tempo de memorizar o conto.

Com **0,0005**, próximo dos 0,0004 do livro, o comportamento é quase idêntico ao treino base: converge até 0,582 no treino e atinge a **melhor validação absoluta de todo o experimento (6,206, na época 7)**, depois sobe.

Com **0,001** e **0,005** o resultado inverte o esperado: as perdas de treino terminam **mais altas** (4,248 e 5,607), não mais baixas. O passo é grande demais e o treino desestabiliza em vez de acelerar. O texto gerado deixa isso evidente — as duas configurações colapsam em sequências de quebras de linha (`"Every effort moves you, I was his"` seguido de dezenas de `\n`), ou seja, o modelo desiste de prever conteúdo e passa a repetir o token mais frequente do corpus.

O tempo é praticamente o mesmo nas quatro (0,9 min, entre 859 e 866 tokens/s), porque a taxa não muda a quantidade de operações: ela só muda o tamanho de cada atualização.

Experimento 5 — Dropout

- Configuração de regularização e sobreajuste (Entregável 4).

O que testar: Se desligar ou aumentar o dropout muda a distância entre a perda de treino e a de validação.

Como fizemos: Três treinos de 10 épocas com `drop_rate` igual a 0,0, 0,1 (o treino base) e 0,2, os valores do grid do livro.

Dropout	Perda treino final	Perda val. final	Diferença	Melhor val. (época)
0,0	0,595	6,372	5,778	6,137 (6)
0,1	0,482	6,381	5,899	6,125 (6)
0,2	0,442	6,459	6,017	6,142 (5)

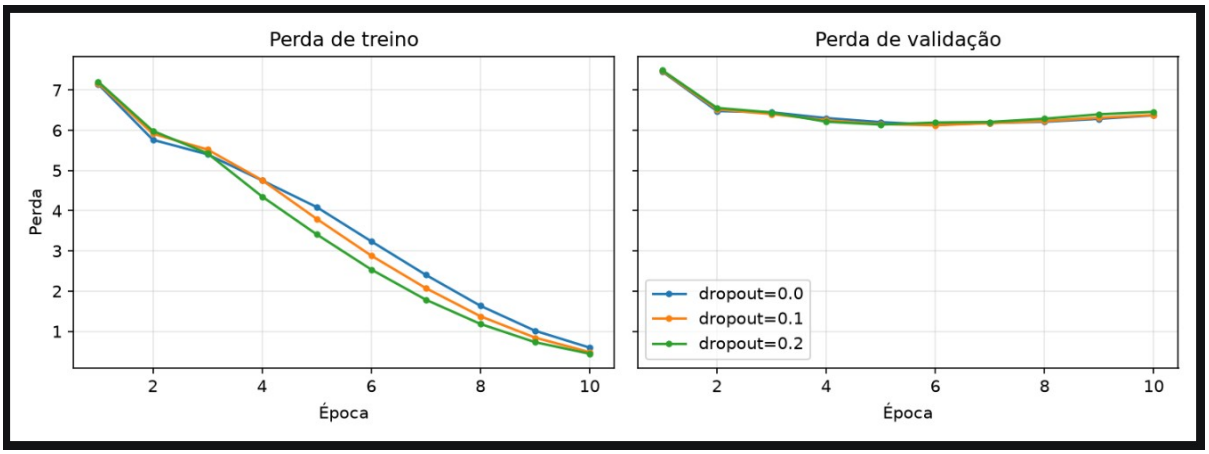


Figura 4 — Perdas por época para cada taxa de dropout (experimentos/dropout.png).

O resultado **contraria a expectativa teórica**. O esperado seria que mais dropout deixasse a perda de treino mais alta (por desligar ativações) e a de validação mais baixa (por regularizar). Aconteceu o oposto nos dois eixos: a perda de treino **caiu** conforme o dropout subiu (0,595 → 0,482 → 0,442) e a de validação **piorou** levemente (6,372 → 6,381 → 6,459). A diferença entre treino e validação, que é a medida do sobreajuste, **aumentou** de 5,778 para 6,017.

Duas leituras possíveis. A primeira é que as diferenças são pequenas — 0,087 entre os extremos na validação — e, com uma única semente por configuração e um lote de validação de apenas 512 tokens, podem ser ruído de medição e não efeito do dropout. A segunda, mais importante, é a conclusão de fundo: **com 9 lotes por época e 4.608 tokens de treino, nenhuma taxa de dropout resolve o problema real, que é a falta de dados**. O dropout regulariza a rede, mas não cria informação nova; se o conjunto de treino é pequeno demais para representar a língua, o modelo vai memorizá-lo de um jeito ou de outro.

Experimento 6 — Estratégias de geração

- Greedy, amostragem, temperatura e top-k com o mesmo contexto (Entregável 5).

O que testar: Como cada estratégia muda a diversidade, a repetição e a coerência do texto, e o que é efeito da estratégia e o que é efeito dos pesos.

Como fizemos: Primeiro reproduzimos o exemplo do livro com um vocabulário de 9 palavras e logits fixos. Depois, no `5,3.py`, geramos 30 tokens a partir de "Every effort moves you" com cada estratégia, 3 sementes por estratégia, no modelo sem treino e no treinado.

Exemplo do livro (vocabulário de 9 palavras)

Temperatura	P(forward)	P(toward)	P(pizza)	Entropia	forward em 1000 sorteios

1	0,572	0,358	0,01%	0,913	582
0,1	0,991	0,009	≈ 0%	0,051	985
5	0,242	0,220	4,30%	1,999	239

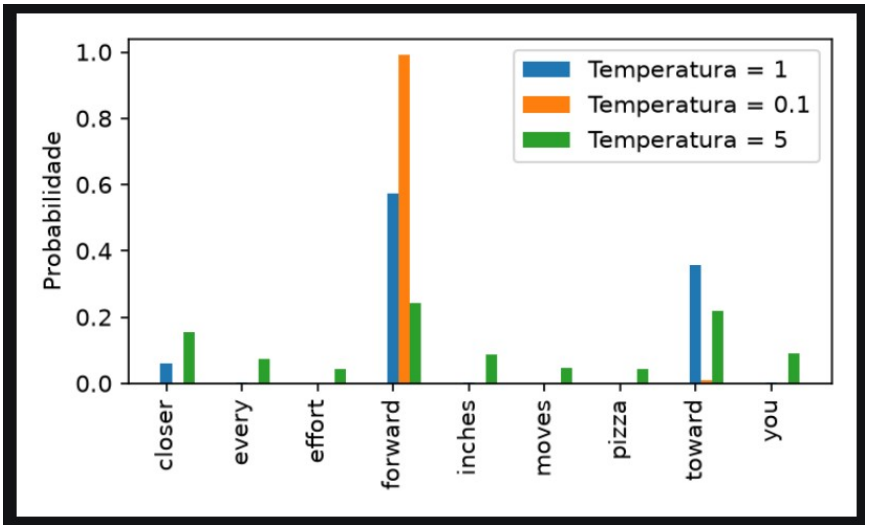


Figura 5 — Distribuição de probabilidade sobre as 9 palavras para T = 1, T = 0,1 e T = 5 (5_3_temperatura.png).

As contagens em 1.000 sorteios acompanham de perto as probabilidades teóricas (0,572 → 582; 0,991 → 985; 0,242 → 239), o que confirma que a amostragem com `torch.multinomial` segue a distribuição esperada. A entropia máxima, com as 9 palavras igualmente prováveis, seria ln 9 ≈ 2,197 — e com T = 5 chegamos a 1,999, bem perto desse teto.

A temperatura baixa deixa a escolha quase determinística; a alta espalha a probabilidade até para palavras sem sentido no contexto, como *pizza*, que passa de 0,01% para 4,3% (exercício 5.1). Com top-k = 3, só *forward*, *toward* e *closer* sobram, com probabilidades 0,578, 0,361 e 0,062.

Mesmo contexto no GPT: modelo sem treino × modelo treinado

Estratégia	Únicos (sem tr.)	Copiado (sem tr.)	Amostras ≠ (trein.)	Únicos (trein.)	Bigramas rep. (trein.)	Copiado (trein.)
Greedy (T=0)	100,0%	1,0	1	86,7%	3,45%	30,0
Amostragem pura (T=1)	100,0%	0,7	3	90,0%	1,15%	8,7
Temperatura baixa (T=0,5)	100,0%	0,7	2	87,8%	2,30%	26,3
Temperatura alta (T=1,5)	100,0%	0,7	3	100,0%	0,00%	1,3
Top-k=1 (T=1)	100,0%	1,0	1	86,7%	3,45%	30,0
Top-k=5 (T=1)	100,0%	0,7	3	81,1%	6,90%	11,0
Top-k=50 (T=1)	98,9%	0,3	3	84,4%	3,45%	9,3
Livro: top-k=25, T=1,4	100,0%	0,7	3	86,7%	0,00%	6,3

A coluna "copiado" é o maior trecho, em tokens, reproduzido literalmente do The Verdict, e é a mais reveladora. No modelo treinado, o **greedy copia 30 tokens de 30** — a geração inteira é um trecho literal do conto. A temperatura alta derruba isso para 1,3 tokens, e o top-k = 25 com T = 1,4 (a combinação do livro) fica em 6,3: diversidade sem perder completamente a ligação com o texto.

Top-k = 1 produz exatamente o mesmo texto do greedy, nas três sementes, confirmando o exercício 5.3: restringir a escolha ao único token mais provável equivale a não amostrar.

No modelo **sem treino**, os 100% de tokens únicos e 0% de bigramas repetidos não indicam "boa diversidade": eles indicam que a distribuição é quase plana e qualquer token é tão provável quanto outro. A entropia confirma — 10,66 com T = 1 no modelo sem treino, contra 3,96 no treinado. Nenhuma estratégia produz texto com sentido quando os pesos não aprenderam nada.

Experimento 7 — Salvar, carregar e retomar o treino

- Salvamento e carregamento dos pesos (Entregável 6).

O que testar: Se o modelo carregado é idêntico ao salvo e se o treino pode ser retomado a partir do checkpoint.

Como fizemos: O `5,2.py` salva `model_and_optimizer.pth` (pesos, estado do AdamW e as configurações). O `5,4.py` salva só os pesos em `model.pth`, carrega em um modelo novo criado com outra semente, compara tensor por tensor, compara a perda e o texto gerado e depois retoma o treino por mais 1 época (exercício 5.4 do livro).

Verificação	Resultado
Tamanho de model.pth	`622,6 MB  (esperado ≈ 623 MiB)`
Tamanho de model_and_optimizer.pth	`1.861,9 MB  (esperado ≈ 1,8 GiB)`
Tensores no state_dict	`173`
Pesos iguais ANTES do load	`False  (modelo novo, outra semente)`
Todos os tensores iguais APÓS o load	`True`
Perda de validação: original × carregado	`6,381409 × 6,381409`
Texto gerado idêntico	`True`
Retomada: perda de treino	`0,4823 → 0,2919`
Retomada: perda de validação	`6,3814 → 6,5274`
Passos do AdamW	`90 → 99`

O arquivo com o otimizador tem cerca de três vezes o tamanho do arquivo só com os pesos, porque o AdamW guarda duas médias móveis (`exp_avg` e `exp_avg_sq`) para cada parâmetro. Elas são necessárias para retomar o treino de onde parou: sem elas, o otimizador recomeçaria as médias do zero.

A verificação "pesos iguais antes do load = False" é o controle do teste: o modelo novo foi criado com outra semente, então começa diferente. Depois do `load_state_dict`, todos os 173 tensores ficam idênticos, a perda de validação bate na sexta casa decimal e o texto gerado é o mesmo caractere por caractere.

A retomada por mais uma época mostra o comportamento já conhecido: a perda de treino cai (0,4823 → 0,2919) e a de validação sobe (6,3814 → 6,5274). O contador de passos do AdamW vai de 90 para 99, confirmando que o otimizador continuou de onde parou em vez de recomeçar.

Experimento 8 — Extra: pesos pré-treinados da OpenAI

O que testar: O que muda quando o mesmo `GPTModel` recebe pesos treinados em um volume muito maior de texto.

Como fizemos: O `5,5.py` baixa os pesos do GPT-2 124M da OpenAI já convertidos para PyTorch pelo autor do livro (a alternativa que o repositório indica para Windows, já que o caminho original depende do TensorFlow, que não roda no Python 3.14), carrega com `context_length = 1024` e `qkv_bias = True`, gera texto e calcula as perdas no The Verdict (exercício 5.5).

Modelo	Perda treino	Perda validação	Perplexidade val.
GPT-2 124M da OpenAI	3,7548	3,5566	35,0
Nosso modelo (10 épocas)	0,4823	6,3814	590,8

O modelo carregado tem 163.037.184 parâmetros, mas `tok_emb` e `out_head` recebem a mesma matriz (weight tying), então os parâmetros únicos são 124.439.808: os ≈ 124M do nome, como calculado na Sprint 4 (a diferença de 27.648 vem do `qkv_bias`).

A comparação é o resultado mais instrutivo da sprint. Nosso modelo tem perda de **treino** muito menor (0,48 contra 3,75) — ele conhece o The Verdict melhor que o GPT-2, porque decorou o conto. Mas na **validação**, que é o trecho final do conto que nenhum dos dois viu em treino, a situação se inverte: 3,56 do GPT-2 contra 6,38 nosso. Em perplexidade, o GPT-2 hesita entre ~35 tokens, o nosso entre ~591.

O texto gerado diz a mesma coisa sem precisar de números. O GPT-2 produz *"Every effort moves you forward. The first step is to understand the importance of your work."* — uma frase original, coerente e gramatical. O nosso produz *"Every effort moves you?" "Yes--quite insensible to the irony…"*, que é uma transcrição literal do conto.

Resumo comparativo

Experimento	Resultado principal
1. Modelo sem treino	perda 10,99 ≈ ln(50.257) = 10,82: o modelo chuta entre todo o vocabulário
2. Treino base	0,4823 (treino) × 6,3814 (validação) após 10 épocas, em 52,2 s
3. Épocas	menor validação na época 6 (6,1247); de 10 a 25 o texto não muda mais
4. Taxa de aprendizado	0,0005 dá a melhor validação (6,206); 0,001 e 0,005 desestabilizam o treino
5. Dropout	não reduziu o sobreajuste — a diferença subiu de 5,778 (0,0) para 6,017 (0,2)
6. Geração	top-k = 25 com T = 1,4: diversidade sem copiar o conto

	(6,3 tokens contra 30 do greedy)
7. Pesos	modelo carregado idêntico ao salvo (173 tensores); treino retomado do passo 90
8. Pesos da OpenAI	validação 3,5566 contra 6,3814 nossa, sem nunca ter visto o texto